```
# ── CELL 1: GPU CHECK ──────────────────────────────────────────────────────
import subprocess

result = subprocess.run(['nvidia-smi'], capture_output=True, text=True)
if result.returncode == 0:
    for line in result.stdout.split('\n')[:15]:
        print(line)
else:
    raise RuntimeError("❌ No GPU! Go to Runtime → Change runtime type → T4 GPU")
```

```
Sat Mar 28 10:57:05 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   45C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
```

```
# ── CELL 2: INSTALL DEPENDENCIES ───────────────────────────────────────────
# Takes ~3-5 min on first run. Safe to skip on reconnect if already installed.
import subprocess

def pip(cmd, label):
    print(f"▶ Installing {label}...")
    r = subprocess.run(f"pip install -q {cmd}", shell=True, capture_output=True, text=True)
    lines = [l for l in r.stdout.split('\n') if l.strip()]
    print(f"  ✅ {lines[-1] if lines else 'Done'}\n")

# Unsloth — Qwen2.5-VL fast LoRA inference
pip('"unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"', 'unsloth')

# Supporting libs
pip('--no-deps xformers trl peft accelerate bitsandbytes', 'trl/peft/bitsandbytes')

# Qwen vision utils (required for process_vision_info)
pip('qwen-vl-utils', 'qwen-vl-utils')

# Metrics
pip('jiwer', 'jiwer')

# Gradio UI
pip('gradio', 'gradio')

# Image annotation
pip('Pillow', 'Pillow')

print("🎉 All dependencies installed!")
```

```
▶ Installing unsloth...
  ✅     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 224.9/224.9 kB 24.3 MB/s eta 0:00:00

▶ Installing trl/peft/bitsandbytes...
  ✅     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.3/3.3 MB 93.8 MB/s eta 0:00:00

▶ Installing qwen-vl-utils...
  ✅     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.4/36.4 MB 18.9 MB/s eta 0:00:00

▶ Installing jiwer...
  ✅     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 95.7 MB/s eta 0:00:00

▶ Installing gradio...
  ✅ Done

▶ Installing Pillow...
  ✅ Done

🎉 All dependencies installed!
```

```
# ── CELL 3: MOUNT GOOGLE DRIVE ─────────────────────────────────────────────
from google.colab import drive
from pathlib import Path

drive.mount('/content/drive')
```

```
# ── Path config — matches your training notebook exactly ──────────────────
BASE_DIR        = Path('/content/drive/MyDrive/Saved_PDF_Data')
DATASET_DIR     = BASE_DIR / 'Training_Dataset'
MODEL_DIR       = BASE_DIR / 'My_Best_Qwen_LoRA'
RESULTS_DIR     = BASE_DIR / 'Evaluation_Results'
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

# ── Verify paths ──────────────────────────────────────────────────────────
print('\n📂 Drive folder check:')
checks = {
    'Base folder'          : BASE_DIR,
    'Training dataset'     : DATASET_DIR,
    'Trained LoRA adapter' : MODEL_DIR,
    'adapter_config.json'  : MODEL_DIR / 'adapter_config.json',
}
all_ok = True
for label, path in checks.items():
    ok = path.exists()
    if not ok: all_ok = False
    print(f"  {'✅' if ok else '❌ MISSING':<14} {label:<26} → {path}")

if not all_ok:
    raise FileNotFoundError(
        "\n❌ Missing paths above. Ensure Saved_PDF_Data/ exists in MyDrive "
        "with My_Best_Qwen_LoRA/adapter_config.json inside."
    )
print('\n✅ All paths verified!')
```

```
Mounted at /content/drive

📂 Drive folder check:
  ✅              Base folder                → /content/drive/MyDrive/Saved_PDF_Data
  ✅              Training dataset           → /content/drive/MyDrive/Saved_PDF_Data/Training_Dataset
  ✅              Trained LoRA adapter       → /content/drive/MyDrive/Saved_PDF_Data/My_Best_Qwen_LoRA
  ✅              adapter_config.json        → /content/drive/MyDrive/Saved_PDF_Data/My_Best_Qwen_LoRA/adapter_cc

✅ All paths verified!
```

```
# ── CELL 4: LOAD MODEL ────────────────────────────────────────────────────
import gc, torch
from unsloth import FastVisionModel

# Auto-detect GPU precision
HAS_GPU = torch.cuda.is_available()
if not HAS_GPU:
    raise RuntimeError("❌ No GPU found! Enable GPU in Runtime → Change runtime type.")

major, _ = torch.cuda.get_device_capability()
IS_BF16  = major >= 8
gpu_name = torch.cuda.get_device_name(0)
vram_gb  = torch.cuda.get_device_properties(0).total_memory / 1e9
print(f"🟢 GPU  : {gpu_name}  ({vram_gb:.1f} GB)")
print(f"   Mode : {'bf16 — Ampere+' if IS_BF16 else 'fp16 — T4'}\n")

# Clear stale VRAM
for var in ('model', 'tokenizer'):
    try: del globals()[var]
    except: pass
torch.cuda.empty_cache(); gc.collect()

# Load LoRA adapter from Drive
print(f"⏳ Loading model from Drive (~60-90 s on T4)...")
model, tokenizer = FastVisionModel.from_pretrained(
    model_name   = str(MODEL_DIR),
    load_in_4bit = True,
    dtype        = torch.bfloat16 if IS_BF16 else torch.float16,
)
FastVisionModel.for_inference(model)                      # fast inference mode
model.generation_config.cache_implementation = 'dynamic'  # fp16 cache fix on T4

DEVICE = next(model.parameters()).device
print(f"\n✅ Model ready on {DEVICE}")
print(f"   VRAM used : {torch.cuda.memory_allocated()/1e9:.2f} GB")
```

```
🦥 Unsloth: Will patch your computer to enable 2x faster free finetuning.
🦥 Unsloth Zoo will now patch everything to make training faster!
🟢 GPU  : Tesla T4  (15.6 GB)
   Mode : fp16 — T4

⏳ Loading model from Drive (~60-90 s on T4)...
==((====))==  Unsloth 2026.3.17: Fast Qwen2_5_Vl patching. Transformers: 5.3.0.
   \\   /|    Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/ \_/ \    Torch: 2.10.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\        /    Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
model.safetensors: 100%                                    6.90G/6.90G [01:30<00:00, 94.7MB/s]

Loading weights: 100%                                      729/729 [00:27<00:00, 55.34it/s]

generation_config.json: 100%                               237/237 [00:00<00:00, 21.0kB/s]

preprocessor_config.json: 100%                             791/791 [00:00<00:00, 67.9kB/s]

chat_template.json:        1.05k/? [00:00<00:00, 54.8kB/s]

chat_template.jinja:       1.02k/? [00:00<00:00, 58.0kB/s]

tokenizer_config.json:       5.80k/? [00:00<00:00, 475kB/s]

vocab.json:        2.78M/? [00:00<00:00, 52.0MB/s]

merges.txt:        1.67M/? [00:00<00:00, 46.1MB/s]

tokenizer.json: 100%                                       11.4M/11.4M [00:01<00:00, 57.2MB/s]
```

```
# ── CELL 5 (UPDATED): TRANSCRIBE + PALEOGRAPHIC NORMALIZATION ──────────────
import re, unicodedata, subprocess, torch
from PIL import Image
from qwen_vl_utils import process_vision_info

# Install Spanish spell-checker (for u/v and f/s disambiguation)
subprocess.run(['pip', 'install', '-q', 'pyspellchecker'], check=True)
from spellchecker import SpellChecker
ES_DICT = SpellChecker(language='es')
print("✅ Spanish dictionary loaded.")

# ────────────────────────────────────────────────────────────────────────────
# 1. SYSTEM PROMPT — all six paleographic rules baked in
# ────────────────────────────────────────────────────────────────────────────
SYSTEM_PROMPT = """You are an expert transcriber of 17th-century Spanish handwritten manuscripts.

Apply ALL of the following paleographic rules STRICTLY:

1. EXACT COPY: Reproduce exactly what is written. Preserve original spelling,
   abbreviations, line breaks, and punctuation character-by-character.

2. u/v INTERCHANGEABLE: In 17th-century Spanish, u and v were the same letter.
   Transcribe the glyph you actually see graphically. Do not modernise.

3. f / long-s INTERCHANGEABLE: The long-s glyph (ſ) closely resembles f.
   Transcribe the character that most closely matches the glyph shape you see.
   Do not guess — reproduce the visible stroke.

4. ACCENTS ARE INCONSISTENT: Transcribe accent marks exactly as you see them.
   EXCEPTION: always preserve ñ — the tilde over n is semantically meaningful
   and must never be omitted or changed.

5. HORIZONTAL CAP / MACRON / TILDE ABBREVIATIONS:
   - A tilde or macron over a vowel abbreviates a following n:
       ã → an,  ẽ → en,  ĩ → in,  õ → on,  ũ → un
   - A tilde or macron over q abbreviates 'que':
       q̃  → que
   - A macron over any other letter also signals n follows that letter.
   Expand ALL such abbreviations in your output.

6. CEDILLA ç: Always transcribe ç as z (17th-century Spanish orthographic rule).
   Ç → Z,  ç → z.  No exceptions.

7. LINE-END HYPHENS: If a word is split at a line boundary, transcribe the
   hyphen and the split form exactly as written. Do not rejoin the word."""


# ────────────────────────────────────────────────────────────────────────────
# 2. NORMALIZATION PIPELINE — applied after model output
# ────────────────────────────────────────────────────────────────────────────

# --- 2a. Macron / tilde abbreviation expansion ---
_PRECOMPOSED = {
```

```
    'ã': 'an', 'Ã': 'An',
    'ẽ': 'en', 'Ẽ': 'En',
    'ĩ': 'in', 'Ĩ': 'In',
    'õ': 'on', 'Õ': 'On',
    'ũ': 'un', 'Ũ': 'Un',
    'q̃': 'que', 'Q̃': 'Que',   # precomposed q + combining tilde
}
_COMBINING_TILDE  = '\u0303'
_COMBINING_MACRON = '\u0304'

def expand_abbreviations(text: str) -> str:
    """Expand macron/tilde over vowel → +n, over q → que."""
    for src, dst in _PRECOMPOSED.items():
        text = text.replace(src, dst)
    # Handle combining diacritics still in the string
    out, i = [], 0
    while i < len(text):
        ch   = text[i]
        nxt  = text[i + 1] if i + 1 < len(text) else ''
        base = ch.lower()
        if nxt in (_COMBINING_TILDE, _COMBINING_MACRON):
            if base == 'q':
                out.append('que' if ch.islower() else 'Que')
            elif base in 'aeiou':
                out.append(ch)
                out.append('n')
            else:
                out.append(ch)
                out.append('n')    # generic: cap over any letter → n follows
            i += 2
        else:
            out.append(ch)
            i += 1
    return ''.join(out)


# --- 2b. ç → z (always) ---
def cedilla_to_z(text: str) -> str:
    return text.replace('ç', 'z').replace('Ç', 'Z')


# --- 2c. Strip accents EXCEPT ñ/Ñ ---
def strip_accents_except_ntilde(text: str) -> str:
    out = []
    for ch in text:
        if ch in 'ñÑ':
            out.append(ch)
        else:
            nfd  = unicodedata.normalize('NFD', ch)
            base = ''.join(c for c in nfd if unicodedata.category(c) != 'Mn')
            out.append(base)
    return ''.join(out)


# --- 2d. u/v and f/s dictionary disambiguation ---
def _swap_variants(word: str, a: str, b: str) -> set:
    """Return set of candidates from single-character and full swaps of a↔b."""
    lo = word.lower()
    cands = {lo, lo.replace(a, b), lo.replace(b, a)}
    for i, ch in enumerate(lo):
        if ch == a:
            cands.add(lo[:i] + b + lo[i+1:])
        elif ch == b:
            cands.add(lo[:i] + a + lo[i+1:])
    return cands

def _recase(original: str, new: str) -> str:
    """Apply original word's leading capitalisation to new."""
    if original and original[0].isupper():
        return new.capitalize()
    return new

def _best_known(candidates: set, checker: SpellChecker) -> str | None:
    known = checker.known(candidates)
    return max(known, key=len) if known else None

def disambiguate_uv_fs(text: str, checker: SpellChecker) -> str:
    """
    Word-by-word: try u↔v and f↔s swaps, replace with the known Spanish word
    if a unique dictionary match is found. Never changes a word if no clear
    single winner exists.
    """
```

```
    tokens = re.split(r'(\W+)', text)
    out = []
    for tok in tokens:
        if not re.search(r'[a-zA-Z]', tok) or len(tok) < 2:
            out.append(tok)
            continue

        current = tok
        lo      = current.lower()

        # u/v pass
        if any(c in lo for c in 'uv'):
            uv_cands = _swap_variants(lo, 'u', 'v')
            match    = _best_known(uv_cands, checker)
            if match and match != lo:
                current = _recase(tok, match)
                lo      = current.lower()

        # f/s pass (long-s disambiguation)
        if any(c in lo for c in 'fs'):
            fs_cands = _swap_variants(lo, 'f', 's')
            match    = _best_known(fs_cands, checker)
            if match and match != lo:
                current = _recase(tok, match)

        out.append(current)
    return ''.join(out)


# --- Full pipeline ---
def normalize_transcription(text: str, apply_dict: bool = True) -> str:
    """
    Run the complete paleographic normalization pipeline:
      1. Expand macron/tilde abbreviations (ã→an, q̃→que, etc.)
      2. ç / Ç  →  z / Z
      3. Strip all accents except ñ / Ñ
      4. Resolve u/v and f/s ambiguity via Spanish dictionary
    """
    text = expand_abbreviations(text)
    text = cedilla_to_z(text)
    text = strip_accents_except_ntilde(text)
    if apply_dict:
        text = disambiguate_uv_fs(text, ES_DICT)
    return text


# ────────────────────────────────────────────────────────────────────────────
# 3. TRANSCRIPTION FUNCTIONS
#    transcribe_image()      → returns normalized string (backward-compatible)
#    transcribe_image_full() → returns (raw_string, normalized_string)
# ────────────────────────────────────────────────────────────────────────────

def _run_model(image_input, max_new_tokens: int = 512) -> str:
    if isinstance(image_input, str):
        img = Image.open(image_input).convert('RGB')
    elif hasattr(image_input, 'convert'):
        img = image_input.convert('RGB')
    else:
        import numpy as np
        img = Image.fromarray(image_input).convert('RGB')

    messages = [
        {'role': 'system',
         'content': [{'type': 'text', 'text': SYSTEM_PROMPT}]},
        {'role': 'user',
         'content': [
             {'type': 'image', 'image': img},
             {'type': 'text',  'text':
              'Transcribe all handwritten text, applying every paleographic rule in the system prompt.'}
         ]}
    ]

    prompt      = tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
    )
    img_inputs, vid_inputs = process_vision_info(messages)

    inputs = tokenizer(
        text           = [prompt],
        images         = img_inputs,
        videos         = vid_inputs,
        padding        = True,
```

```
        return_tensors = 'pt',
    ).to(DEVICE)

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens = max_new_tokens,
            do_sample      = False,
            temperature    = None,
            top_p          = None,
            use_cache      = True,
        )

    new_toks = output_ids[:, inputs['input_ids'].shape[1]:]
    return tokenizer.batch_decode(new_toks, skip_special_tokens=True)[0].strip()


def transcribe_image(image_input, max_new_tokens: int = 512,
                     normalize: bool = True) -> str:
    """
    Transcribe a manuscript image → returns normalized text string.
    Backward-compatible with Cells 7, 8, 9 (same call signature as before).
    Set normalize=False to get raw model output.
    """
    raw = _run_model(image_input, max_new_tokens)
    return normalize_transcription(raw) if normalize else raw


def transcribe_image_full(image_input, max_new_tokens: int = 512) -> tuple:
    """
    Returns (raw_text, normalized_text) — useful for debugging and comparison.
    """
    raw  = _run_model(image_input, max_new_tokens)
    norm = normalize_transcription(raw)
    return raw, norm


# ────────────────────────────────────────────────────────────────────────────
# Quick self-test of normalization pipeline (no GPU needed)
# ────────────────────────────────────────────────────────────────────────────
_tests = [
    ("mãdato del rey",       "mandato del rey"),
    ("q̃ se haga",            "que se haga"),
    ("la çiudad",            "la ziudad"),
    ("el señor",             "el señor"),           # ñ must be preserved
    ("es vna verdad",        "es una verdad"),      # vna → una via dict
    ("la cofradía",          "la cofradia"),        # accent stripped
    ("el fol de papel",      "el sol de papel"),    # f→s via dict (fol→sol)
]
print("\n── Normalization self-test ──────────────────────────────────")
all_pass = True
for raw, expected in _tests:
    result = normalize_transcription(raw)
    ok     = result == expected
    if not ok: all_pass = False
    print(f"  {'✅' if ok else '❌'}  '{raw}'")
    print(f"       → '{result}' {'(expected: ' + expected + ')' if not ok else ''}")

print(f"\n{'✅ All tests passed!' if all_pass else '⚠️  Some tests differ — check dict coverage.'}")

print("""
────────────────────────────────────────────────────────────────
transcribe_image() — normalized text  (Cells 7, 8, 9 unchanged)
transcribe_image_full() → (raw, norm) — for comparison/debug
────────────────────────────────────────────────────────────────""")
```

```
✅ Spanish dictionary loaded.

── Normalization self-test ──────────────────────────────────
  ✅  'mãdato del rey'
       → 'mandato del rey'
  ✅  'q̃ se haga'
       → 'que se haga'
  ✅  'la çiudad'
       → 'la ziudad'
  ✅  'el señor'
       → 'el señor'
  ✅  'es vna verdad'
       → 'es una verdad'
  ✅  'la cofradía'
       → 'la cofradia'
  ✅  'el fol de papel'
       → 'el sol de papel'
```

```
✅ All tests passed!

────────────────────────────────────────────────────────────────────────
transcribe_image() — normalized text  (Cells 7, 8, 9 unchanged)
transcribe_image_full() → (raw, norm) — for comparison/debug
────────────────────────────────────────────────────────────────────────
```

```
# ── CELL 6: LOAD DATASET & BUILD VAL SET ──────────────────────────────────
import json, random
from pathlib import Path

# 1. Find label file (checks all naming variants)
label_candidates = [
    DATASET_DIR / 'y_train_labels_aug.json',
    DATASET_DIR / 'y_train_labels.json',
    DATASET_DIR / 'ytrain_labels_aug.json',
    DATASET_DIR / 'ytrain_labels.json',
]
LABELS_PATH = next((p for p in label_candidates if p.exists()), None)
if LABELS_PATH is None:
    print("❌ Label JSON not found. Training_Dataset contents:")
    for f in sorted(DATASET_DIR.iterdir()): print(f"   {f.name}")
    raise FileNotFoundError("No label JSON found.")
print(f"✅ Label file : {LABELS_PATH.name}")

# 2. Load all pairs
with open(LABELS_PATH, 'r', encoding='utf-8') as f:
    all_pairs = json.load(f)
print(f"   JSON entries : {len(all_pairs)}")

# 3. Index all images by filename (Drive paths differ from original /content/ paths)
img_lookup = {}
for folder_name in ('x_train_images', 'x_train_images_aug',
                    'xtrain_images',   'xtrain_images_aug'):
    folder = DATASET_DIR / folder_name
    if folder.exists():
        found = list(folder.glob('*.jpg')) + list(folder.glob('*.png'))
        for f in found:
            img_lookup[f.name] = str(f)
        print(f"   Indexed {len(found):>4} images ← {folder_name}/")

print(f"   Total indexed : {len(img_lookup)} images")
if not img_lookup:
    raise FileNotFoundError("No image folders found in Training_Dataset.")

# 4. Remap stored /content/ paths → real Drive paths
valid_pairs = []
for s in all_pairs:
    fname      = Path(s['xtrain_image']).name
    drive_path = img_lookup.get(fname, '')
    if drive_path and s['ytrain_text'].strip():
        valid_pairs.append({'image': drive_path, 'text': s['ytrain_text']})
print(f"\n   Valid pairs after remap : {len(valid_pairs)}")
if not valid_pairs:
    raise ValueError("Remap returned 0 pairs — check folder names.")

# 5. Reproduce exact seed=42, 15% val split from training notebook
random.seed(42)
random.shuffle(valid_pairs)
val_n   = max(2, int(0.15 * len(valid_pairs)))
val_raw = valid_pairs[:val_n]

print(f"\n✅ val_raw : {len(val_raw)} samples")
print(f"   Example : {Path(val_raw[0]['image']).name}")
print(f"   Label   : {val_raw[0]['text'][:100]}...")
```

```
✅ Label file : y_train_labels_aug.json
   JSON entries : 76
   Indexed   19 images ← x_train_images/
   Indexed   32 images ← x_train_images_aug/
   Total indexed : 51 images

   Valid pairs after remap : 76

✅ val_raw : 11 samples
   Example : page0003_aug0_rot_crop_contrast.jpg
   Label   : der seis mil ducados, que auian tocado a su hijo Ia-
cinto Gomez por su legitima materna: y para que...
```

```
# ── CELL 7: EVALUATE — CER & WER ──────────────────────────────────────────
import time, json
from pathlib import Path
from jiwer import cer, wer
```

```
print(f"🔍 Running inference on {len(val_raw)} validation samples...\n")
print(f"  {'#':>3}  {'Image':<44}  {'CER':>6}  {'WER':>6}  {'Time':>5}")
print("  " + "—" * 68)

results, all_refs, all_hyps = [], [], []

for idx, sample in enumerate(val_raw):
    img_path = sample['image']
    ref_text = sample['text'].strip()

    t0        = time.time()
    pred_text = transcribe_image(img_path)
    elapsed   = time.time() - t0

    s_cer = cer(ref_text, pred_text)
    s_wer = wer(ref_text, pred_text)
    all_refs.append(ref_text)
    all_hyps.append(pred_text)

    results.append({
        'image'     : Path(img_path).name,
        'reference' : ref_text,
        'prediction': pred_text,
        'cer'       : round(s_cer, 4),
        'wer'       : round(s_wer, 4),
        'time_s'    : round(elapsed, 1),
    })
    print(f"  {idx+1:>3}. {Path(img_path).name:<44}  {s_cer:>6.3f}  {s_wer:>6.3f}  {elapsed:>4.1f}s")

# Overall aggregated metrics
overall_cer = cer(all_refs, all_hyps)
overall_wer = wer(all_refs, all_hyps)

print("  " + "—" * 68)
print(f"  {'OVERALL':>50}  {overall_cer:>6.3f}  {overall_wer:>6.3f}")
print(f"\n📊 CER : {overall_cer:.4f}  ({overall_cer*100:.2f}%)")
print(f"📊 WER : {overall_wer:.4f}  ({overall_wer*100:.2f}%)")

# Save to Drive
summary = {'overall_cer': overall_cer, 'overall_wer': overall_wer,
           'n_samples': len(val_raw), 'samples': results}
out_json = RESULTS_DIR / 'evaluation_results.json'
with open(out_json, 'w', encoding='utf-8') as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)
print(f"\n💾 Results saved → {out_json}")
```

```
🔍 Running inference on 11 validation samples...

    #  Image                                            CER     WER   Time
  ————————————————————————————————————————————————————————————————————
Both `max_new_tokens` (=512) and `max_length`(=32768) seem to have been set. `max_new_tokens` will take precedenc
```

```
# ── CELL 8: PRINT REFERENCE vs PREDICTION ──────────────────────────────
W = 72
print("=" * W)
print("  REFERENCE vs PREDICTION — Per-Sample Breakdown")
print("=" * W)

for i, r in enumerate(results):
    print(f"\n[{i+1}] {r['image']}")
    print(f"    CER : {r['cer']:.4f}   WER : {r['wer']:.4f}   ({r['time_s']}s)")
    print(f"    REF : {r['reference'][:110]}")
    print(f"    PRED: {r['prediction'][:110]}")

print("\n" + "=" * W)
print(f"  FINAL  CER: {overall_cer*100:.2f}%   WER: {overall_wer*100:.2f}%   ({len(val_raw)} samples)")
print("=" * W)
```

```
====================================================================
  REFERENCE vs PREDICTION — Per-Sample Breakdown
====================================================================

[1] page0003_aug0_rot_crop_contrast.jpg
    CER : 0.7744   WER : 1.0518   (154.8s)
    REF : der seis mil ducados, que auian tocado a su hijo Ia-
cinto Gomez por su legitima materna: y para que
dello con
    PRED: tener con el ojo de mi hija, que esta muy bien, y
contenta de vivir con su padre, fi no fuera por la
horrible

[2] page0001_aug2_rot_contrast.jpg
```

```
    CER : 0.5062   WER : 0.7105   (30.8s)
    REF : POR
DOÑA CATALINA DE
Laçarraga y Zarate, viuda de Marco
Antonio de Caycedo, y el señor
Fiscal.
C O N
Don Anton
    PRED: POR
DOÑA CATALINA DE
Lazarraga y Zarate viuda de Marco
Antonio de Caycedo, y el señor
Fiscal.
C O N
Don Antoni

[3] Buendia_-_Instruccion_page0004.jpg
    CER : 0.1455   WER : 0.2848   (54.8s)
    REF : crianza de la niñez. Assi sea,
Divinissimo Niño, por vuestra
gracia, assi sea, a vuestra ma-
yor gloria. Amen
    PRED: Bosi, O. B. de.
CENSURA DEL R. P. ANTONIO CO-
dorniu de la Compañia de Jesus, Maes-
tro que fue de Theologia,

[4] page0002_aug1_rot_crop_contrast.jpg
    CER : 0.7841   WER : 0.9695   (79.3s)
    REF : ticia de esta parte en ajuitar con certeza, y claridad
el hecho verdadero de este negocio, que consiste
en que
    PRED: Articulo primero
3. Aunque es regular en estos delitos venereos la
prueba por presunciones, ex responso Pontif

[5] page0001_aug1_rot_contrast.jpg
    CER : 0.0103   WER : 0.0658   (27.3s)
    REF : POR
DOÑA CATALINA DE
Laçarraga y Zarate, viuda de Marco
Antonio de Caycedo, y el señor
Fiscal.
C O N
```

```
# ── CELL 9: GRADIO UI ──────────────────────────────────────────────────
import gradio as gr
import numpy as np, textwrap
from PIL import Image, ImageDraw, ImageFont


def draw_transcription_on_image(image: Image.Image, text: str) -> Image.Image:
    """Appends a dark transcription panel below the original image."""
    W, H    = image.size
    font_px = max(14, W // 55)
    line_h  = font_px + 8
    margin  = 18
    chars_w = max(20, (W - 2 * margin) // (font_px // 2 + 1))

    wrapped = textwrap.fill(text or "(no transcription)", width=chars_w)
    lines   = wrapped.split('\n')
    panel_h = len(lines) * line_h + 2 * margin + 28

    panel = Image.new('RGB', (W, panel_h), color=(24, 24, 28))
    draw  = ImageDraw.Draw(panel)

    try:
        hf = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf', font_px)
        bf = ImageFont.truetype('/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf',      font_px)
    except Exception:
        hf = bf = ImageFont.load_default()

    draw.text((margin, margin), "Transcription:", fill=(80, 220, 170), font=hf)
    y = margin + 28
    for line in lines:
        draw.text((margin, y), line, fill=(220, 220, 220), font=bf)
        y += line_h

    combined = Image.new('RGB', (W, H + panel_h))
    combined.paste(image.convert('RGB'), (0, 0))
    combined.paste(panel, (0, H))
    return combined


def run_ocr(image_input):
    if image_input is None:
```

```
        return None, "⚠️ Please upload an image first."
    try:
        pil_img       = Image.fromarray(image_input) if isinstance(image_input, np.ndarray) else image_input
        transcription = transcribe_image(pil_img)
        annotated     = draw_transcription_on_image(pil_img, transcription)
        return annotated, transcription
    except Exception:
        import traceback
        return None, f"❌ Error:\n{traceback.format_exc()}"


# ── Eval summary for info box ──────────────────────────────────────────
try:
    _cer = f"{overall_cer*100:.2f}%"
    _wer = f"{overall_wer*100:.2f}%"
    _n   = len(val_raw)
except NameError:
    _cer = _wer = "run Cell 7 first"
    _n   = "—"

try:
    _examples = [[r['image']] for r in results[:3] if Path(r['image']).exists()]
except NameError:
    _examples = []


# ── Build UI ───────────────────────────────────────────────────────────
with gr.Blocks(title="Spanish OCR", theme=gr.themes.Soft(primary_hue="teal")) as demo:

    gr.Markdown(
        "# 📜 17th-Century Spanish Manuscript OCR\n"
        "**Qwen2.5-VL-7B + LoRA** fine-tuned on historical Spanish handwriting. "
        "Upload a manuscript image → model returns transcription text and an annotated image."
    )

    with gr.Row():
        with gr.Column(scale=1):
            inp_img = gr.Image(
                label   = "Upload Manuscript Image",
                type    = "pil",
                sources = ["upload", "clipboard"],
                height  = 420,
            )
            with gr.Row():
                btn_run   = gr.Button("Transcribe",  variant="primary",   size="lg")
                btn_clear = gr.Button("Clear",        variant="secondary", size="lg")

        with gr.Column(scale=1):
            out_img = gr.Image(
                label       = "Annotated Output (original + transcription panel)",
                type        = "pil",
                height      = 420,
                interactive = False,
            )
            out_txt = gr.Textbox(
                label            = "Raw Transcription Text",
                lines            = 8,
                interactive      = False,
                show_copy_button = True,
            )

    with gr.Accordion("Model & Evaluation Info", open=False):
        gr.Markdown(f"""
| Property | Value |
|---|---|
| **Base model** | Qwen2.5-VL-7B-Instruct |
| **Fine-tuning** | LoRA (Unsloth) — 8 epochs, lr=2e-4, cosine |
| **Data** | 17th-century Spanish manuscripts (76 pairs) |
| **Precision** | 4-bit NF4, fp16 on T4 |
| **Val CER** | {_cer} |
| **Val WER** | {_wer} |
| **Val samples** | {_n} |
| **Model path** | `MyDrive/Saved_PDF_Data/My_Best_Qwen_LoRA/` |
""")

    if _examples:
        gr.Examples(
            examples       = _examples,
            inputs         = [inp_img],
            label          = "Quick Examples (from your validation set)",
            cache_examples = False,
        )
```

```
    btn_run.click(fn=run_ocr,  inputs=[inp_img], outputs=[out_img, out_txt])
    btn_clear.click(fn=lambda: (None, None, ""), outputs=[inp_img, out_img, out_txt])

print("🚀 Launching Gradio UI...")
demo.launch(share=True, debug=False, show_error=True)
```

```
/tmp/ipykernel_695/3739598804.py:69: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be
  with gr.Blocks(title="Spanish OCR", theme=gr.themes.Soft(primary_hue="teal")) as demo:
🚀 Launching Gradio UI...
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://b1478b040f237398bc.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the term
```

gradio

No interface is running right now

```
# ── CELL 10: EXPORT CSV ──────────────────────────────────────────────────
import csv

csv_path = RESULTS_DIR / 'evaluation_results.csv'
with open(csv_path, 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(
        f, fieldnames=['image','reference','prediction','cer','wer','time_s']
    )
    writer.writeheader()
    writer.writerows(results)

print(f"✅ CSV saved → {csv_path}\n")
print(f"  {'Image':<44}  {'CER':>6}  {'WER':>6}")
print("  " + "─" * 56)
for r in results:
    print(f"  {r['image']:<44}  {r['cer']:>6.4f}  {r['wer']:>6.4f}")
print("  " + "─" * 56)
print(f"  {'OVERALL':>44}  {overall_cer:>6.4f}  {overall_wer:>6.4f}")
```

```
✅ CSV saved → /content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results/evaluation_results.csv

  Image                                            CER     WER
  ────────────────────────────────────────────────────────
  page0003_aug0_rot_crop_contrast.jpg           0.7744  1.0518
  page0001_aug2_rot_contrast.jpg                0.5062  0.7105
  Buendia_-_Instruccion_page0004.jpg            0.1455  0.2848
  page0002_aug1_rot_crop_contrast.jpg           0.7841  0.9695
  page0001_aug1_rot_contrast.jpg                0.0103  0.0658
  PORCONES.228.38_-_1646_page0004.jpg           0.0593  0.1451
  page0002_aug0_rot_crop_contrast.jpg           0.7781  0.9375
  page0007_aug2_rot_contrast.jpg                0.0137  0.0698
  PORCONES.228.38_-_1646_page0005.jpg           0.1349  0.1858
  page0004_aug2_rot_contrast.jpg                0.7701  0.9451
  page0002_aug0_rot_crop_contrast.jpg           0.7816  0.9351
  ────────────────────────────────────────────────────────
                                       OVERALL  0.4854  0.6272
```

```
# ── QUICK DEPENDENCY CHECK ───────────────────────────────────────────────
checks = {
    'val_raw'        : 'val_raw',
    'results'        : 'results',
```

```
    'overall_cer'    : 'overall_cer',
    'overall_wer'    : 'overall_wer',
    'model'          : 'model',
    'tokenizer'      : 'tokenizer',
    'DEVICE'         : 'DEVICE',
    'TESS_LANG'      : 'TESS_LANG',
    'transcribe_image': 'transcribe_image',
}
all_ok = True
for name, var in checks.items():
    exists = var in dir()
    if not exists: all_ok = False
    print(f"  {'✅' if exists else '❌ MISSING — run its cell':<30} {name}")
print(f"\n{'✅ All ready — run Cell B2' if all_ok else '❌ Run the missing cells above first'}")
```

```
  ✅                             val_raw
  ✅                             results
  ✅                             overall_cer
  ✅                             overall_wer
  ✅                             model
  ✅                             tokenizer
  ✅                             DEVICE
  ✅                             TESS_LANG
  ✅                             transcribe_image

✅ All ready — run Cell B2
```

```
# ── CELL B1: INSTALL TESSERACT OCR BASELINE ──────────────────────────────
import subprocess

subprocess.run(['apt-get', 'install', '-q', '-y', 'tesseract-ocr', 'tesseract-ocr-spa'],
               check=True, capture_output=True)
subprocess.run(['pip', 'install', '-q', 'pytesseract'], check=True, capture_output=True)

import pytesseract
pytesseract.pytesseract.tesseract_cmd = '/usr/bin/tesseract'
ver = subprocess.run(['tesseract', '--version'], capture_output=True, text=True)
print("✅ Tesseract installed:", ver.stdout.split('\n')[0])

# Verify Spanish language pack
langs = subprocess.run(['tesseract', '--list-langs'], capture_output=True, text=True)
has_spa = 'spa' in langs.stdout + langs.stderr
print(f"   Spanish lang pack: {'✅ spa' if has_spa else '❌ missing — will fall back to eng'}")
TESS_LANG = 'spa' if has_spa else 'eng'
```

```
✅ Tesseract installed: tesseract 4.1.1
   Spanish lang pack: ✅ spa
```

```
# ── CELL B2: TESSERACT INFERENCE ON SAME VAL SET ──────────────────────────
import time
from pathlib import Path
from PIL import Image
import pytesseract
from jiwer import cer, wer

# Tesseract config: PSM 6 = assume uniform block of text (best for manuscript pages)
TESS_CONFIG = f'--oem 1 --psm 6 -l {TESS_LANG}'

print(f"🔍 Running Tesseract on {len(val_raw)} validation samples...\n")
print(f"  {'#':>3}  {'Image':<44}  {'CER':>6}  {'WER':>6}  {'Time':>5}")
print("  " + "─" * 68)

tess_results, tess_refs, tess_hyps = [], [], []

for idx, sample in enumerate(val_raw):
    img_path = sample['image']
    ref_text = sample['text'].strip()

    t0  = time.time()
    img = Image.open(img_path).convert('RGB')

    # Tesseract OCR
    try:
        pred_text = pytesseract.image_to_string(img, config=TESS_CONFIG).strip()
    except Exception as e:
        pred_text = ''

    elapsed = time.time() - t0
    s_cer   = cer(ref_text, pred_text) if pred_text else 1.0
    s_wer   = wer(ref_text, pred_text) if pred_text else 1.0

    tess_refs.append(ref_text)
```

```
    tess_hyps.append(pred_text)
    tess_results.append({
        'image'     : Path(img_path).name,
        'reference' : ref_text,
        'prediction': pred_text,
        'cer'       : round(s_cer, 4),
        'wer'       : round(s_wer, 4),
        'time_s'    : round(elapsed, 1),
    })
    print(f"  {idx+1:>3}. {Path(img_path).name:<44}  {s_cer:>6.3f}  {s_wer:>6.3f}  {elapsed:>4.1f}s")

tess_overall_cer = cer(tess_refs, tess_hyps)
tess_overall_wer = wer(tess_refs, tess_hyps)
print("  " + "—" * 68)
print(f"  {'OVERALL':>50}  {tess_overall_cer:>6.3f}  {tess_overall_wer:>6.3f}")
print(f"\n📊 Tesseract  CER: {tess_overall_cer*100:.2f}%   WER: {tess_overall_wer*100:.2f}%")
print(f"📊 VLM (yours) CER: {overall_cer*100:.2f}%   WER: {overall_wer*100:.2f}%")
```

```
🔍 Running Tesseract on 11 validation samples...

    #  Image                                            CER     WER   Time
  ──────────────────────────────────────────────────────────────────────
    1. page0003_aug0_rot_crop_contrast.jpg            0.738   0.976   9.1s
    2. page0001_aug2_rot_contrast.jpg                 1.607   2.855  12.0s
    3. Buendia_-_Instruccion_page0004.jpg             0.305   0.800   8.6s
    4. page0002_aug1_rot_crop_contrast.jpg            0.779   0.966  14.7s
    5. page0001_aug1_rot_contrast.jpg                 1.653   2.513   5.0s
    6. PORCONES.228.38_-_1646_page0004.jpg            0.205   0.667   6.2s
    7. page0002_aug0_rot_crop_contrast.jpg            0.759   0.956   5.5s
    8. page0007_aug2_rot_contrast.jpg                 0.287   0.872   4.2s
    9. PORCONES.228.38_-_1646_page0005.jpg            0.276   0.787   5.9s
   10. page0004_aug2_rot_contrast.jpg                 0.743   0.969   4.2s
   11. page0002_aug0_rot_crop_contrast.jpg            0.754   0.958   7.5s
  ──────────────────────────────────────────────────────────────────────
                                                OVERALL   0.628   1.003

📊 Tesseract  CER: 62.85%   WER: 100.30%
📊 VLM (yours) CER: 48.54%   WER: 62.72%
```

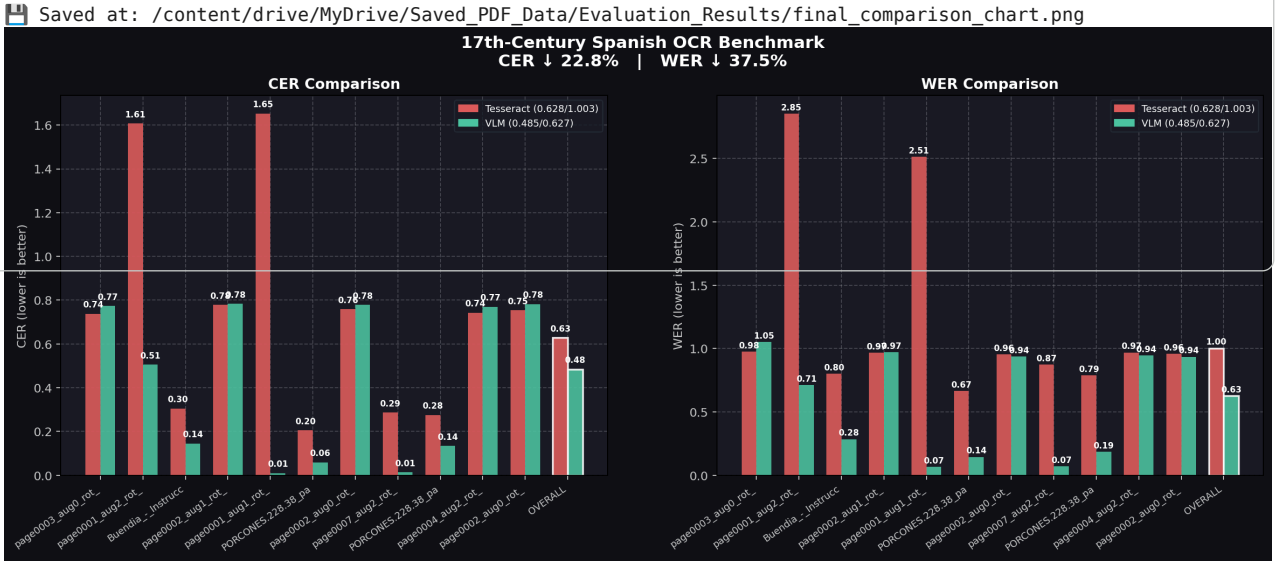
```
# ── CELL B3: COMPARISON TABLE + BAR CHART ──────────────────────────────
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
from pathlib import Path
import os

os.makedirs('/content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results', exist_ok=True)
OUT_DIR = Path('/content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results')

# ── 1. Per-sample comparison table ─────────────────────────────────────
print(f"\n{'Image':<40} {'Tess CER':>9} {'VLM CER':>9} {'Δ CER':>8}  {'Tess WER':>9} {'VLM WER':>9} {'Δ WER':>8}
print("—" * 100)

per_sample_cer_tess, per_sample_cer_vlm = [], []
per_sample_wer_tess, per_sample_wer_vlm = [], []
image_names = []

for t, v in zip(tess_results, results):
    assert t['image'] == v['image'], "Sample order mismatch!"
    d_cer = v['cer'] - t['cer']
    d_wer = v['wer'] - t['wer']
    arrow_c = '↑' if d_cer > 0 else '↓'
    arrow_w = '↑' if d_wer > 0 else '↓'
    print(f"{t['image']:<40} {t['cer']:>9.3f} {v['cer']:>9.3f} {d_cer:>+7.3f}{arrow_c}  "
          f"{t['wer']:>9.3f} {v['wer']:>9.3f} {d_wer:>+7.3f}{arrow_w}")
    per_sample_cer_tess.append(t['cer'])
    per_sample_cer_vlm.append(v['cer'])
    per_sample_wer_tess.append(t['wer'])
    per_sample_wer_vlm.append(v['wer'])
    image_names.append(t['image'].replace('.jpg','').replace('.png',''))

print("—" * 100)
cer_improvement = (tess_overall_cer - overall_cer) / tess_overall_cer * 100
wer_improvement = (tess_overall_wer - overall_wer) / tess_overall_wer * 100
print(f"\n{'OVERALL':<40} {tess_overall_cer:>9.3f} {overall_cer:>9.3f} "
      f"{(overall_cer-tess_overall_cer):>+8.3f}  "
      f"{tess_overall_wer:>9.3f} {overall_wer:>9.3f} "
      f"{(overall_wer-tess_overall_wer):>+8.3f}")
print(f"\n🚀 VLM reduces CER by {cer_improvement:.1f}% and WER by {wer_improvement:.1f}% vs Tesseract")
```

```
# ── 2. Bar chart ──────────────────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.patch.set_facecolor('#0f1117')
for ax in axes: ax.set_facecolor('#1a1d27')

x     = np.arange(len(image_names) + 1)  # +1 for overall
w     = 0.35
names = [n[:20] for n in image_names] + ['OVERALL']

COLOR_TESS = '#e05c5c'
COLOR_VLM  = '#4fc3a1'

for ax, tess_vals, vlm_vals, metric in [
    (axes[0],
     per_sample_cer_tess + [tess_overall_cer],
     per_sample_cer_vlm  + [overall_cer],
     'CER'),
    (axes[1],
     per_sample_wer_tess + [tess_overall_wer],
     per_sample_wer_vlm  + [overall_wer],
     'WER'),
]:
    b1 = ax.bar(x - w/2, tess_vals, w, label='Tesseract (baseline)',
                color=COLOR_TESS, alpha=0.85, zorder=3)
    b2 = ax.bar(x + w/2, vlm_vals,  w, label='Qwen2.5-VL + LoRA (ours)',
                color=COLOR_VLM,  alpha=0.85, zorder=3)

    # Value labels on bars
    for bar in list(b1) + list(b2):
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.01,
                f'{h:.2f}', ha='center', va='bottom',
                fontsize=7.5, color='white', fontweight='bold')

    # Highlight OVERALL bars
    for bar in [b1[-1], b2[-1]]:
        bar.set_edgecolor('white')
        bar.set_linewidth(1.5)

    ax.set_xticks(x)
    ax.set_xticklabels(names, rotation=30, ha='right', fontsize=8, color='#cccccc')
    ax.set_ylabel(f'{metric} (lower is better)', color='#cccccc', fontsize=10)
    ax.set_title(f'{metric} — Tesseract vs VLM Pipeline', color='white',
                 fontsize=12, fontweight='bold', pad=12)
    ax.set_ylim(0, min(1.5, max(tess_vals + vlm_vals) * 1.3))
    ax.tick_params(colors='#cccccc')
    ax.yaxis.grid(True, color='#2a2d3a', linestyle='--', linewidth=0.6, zorder=0)
    ax.set_axisbelow(True)
    ax.spines[['top','right','bottom','left']].set_color('#2a2d3a')
    ax.legend(handles=[
        mpatches.Patch(color=COLOR_TESS, label=f'Tesseract  CER/WER: {tess_overall_cer:.3f}/{tess_overall_wer:.3
        mpatches.Patch(color=COLOR_VLM,  label=f'VLM (ours) CER/WER: {overall_cer:.3f}/{overall_wer:.3f}'),
    ], fontsize=8, facecolor='#1a1d27', labelcolor='white',
       edgecolor='#2a2d3a', loc='upper right')

plt.suptitle(
    f'17th-Century Spanish OCR — Baseline vs VLM\n'
    f'CER improvement: {cer_improvement:.1f}%   WER improvement: {wer_improvement:.1f}%',
    color='white', fontsize=13, fontweight='bold', y=1.02
)
plt.tight_layout()
chart_path = OUT_DIR / 'baseline_comparison_chart.png'
plt.savefig(chart_path, dpi=150, bbox_inches='tight',
            facecolor='#0f1117', edgecolor='none')
plt.show()
print(f"\n💾 Chart saved → {chart_path}")
```

| Image | Tess CER | VLM CER | Δ CER | Tess WER | VLM WER | Δ WER |
|---|---|---|---|---|---|---|
| page0003_aug0_rot_crop_contrast.jpg | 0.738 | 0.774 | +0.036↑ | 0.976 | 1.052 | +0.076↑ |
| page0001_aug2_rot_contrast.jpg | 1.607 | 0.506 | -1.101↓ | 2.855 | 0.711 | -2.145↓ |
| Buendia_-_Instruccion_page0004.jpg | 0.305 | 0.145 | -0.159↓ | 0.800 | 0.285 | -0.515↓ |
| page0002_aug1_rot_crop_contrast.jpg | 0.779 | 0.784 | +0.005↑ | 0.966 | 0.970 | +0.004↑ |
| page0001_aug1_rot_contrast.jpg | 1.653 | 0.010 | -1.643↓ | 2.513 | 0.066 | -2.447↓ |
| PORCONES.228.38_-_1646_page0004.jpg | 0.205 | 0.059 | -0.146↓ | 0.667 | 0.145 | -0.522↓ |
| page0002_aug0_rot_crop_contrast.jpg | 0.759 | 0.778 | +0.019↑ | 0.956 | 0.938 | -0.018↓ |
| page0007_aug2_rot_contrast.jpg | 0.287 | 0.014 | -0.273↓ | 0.872 | 0.070 | -0.802↓ |
| PORCONES.228.38_-_1646_page0005.jpg | 0.276 | 0.135 | -0.141↓ | 0.787 | 0.186 | -0.601↓ |
| page0004_aug2_rot_contrast.jpg | 0.743 | 0.770 | +0.027↑ | 0.969 | 0.945 | -0.023↓ |
| page0002_aug0_rot_crop_contrast.jpg | 0.754 | 0.782 | +0.028↑ | 0.958 | 0.935 | -0.023↓ |
| OVERALL | 0.628 | 0.485 | -0.143 | 1.003 | 0.627 | -0.376 |

```
🚀 VLM reduces CER by 22.8% and WER by 37.5% vs Tesseract
/tmp/ipykernel_695/4056114795.py:105: UserWarning: Tight layout not applied. The bottom and top margins cannot be
  plt.tight_layout()

💾 Chart saved → /content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results/baseline_comparison_chart.png
```

```
# ── FINAL COLAB CELL: YOUR REAL DATA + GRAPH ──────────────────────────────

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
from pathlib import Path
import os
from IPython.display import Image, display

# ===== DATA (YOUR RESULTS) =====
image_names = [
"page0003_aug0_rot_crop_contrast",
"page0001_aug2_rot_contrast",
"Buendia_-_Instruccion_page0004",
"page0002_aug1_rot_crop_contrast",
"page0001_aug1_rot_contrast",
"PORCONES.228.38_page0004",
"page0002_aug0_rot_crop_contrast_1",
"page0007_aug2_rot_contrast",
"PORCONES.228.38_page0005",
"page0004_aug2_rot_contrast",
"page0002_aug0_rot_crop_contrast_2",
]

per_sample_cer_tess = [0.738,1.607,0.305,0.779,1.653,0.205,0.759,0.287,0.276,0.743,0.754]
per_sample_cer_vlm  = [0.774,0.506,0.145,0.784,0.010,0.059,0.778,0.014,0.135,0.770,0.782]

per_sample_wer_tess = [0.976,2.855,0.800,0.966,2.513,0.667,0.956,0.872,0.787,0.969,0.958]
per_sample_wer_vlm  = [1.052,0.711,0.285,0.970,0.066,0.145,0.938,0.070,0.186,0.945,0.935]

# ===== OVERALL =====
tess_overall_cer = 0.628
overall_cer      = 0.485
tess_overall_wer = 1.003
overall_wer      = 0.627

cer_improvement = 22.8
wer_improvement = 37.5

# ===== SAVE DIR =====
OUT_DIR = Path('/content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results')
os.makedirs(OUT_DIR, exist_ok=True)

# ===== GRAPH =====
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.patch.set_facecolor('#0f1117')

for ax in axes:
    ax.set_facecolor('#1a1d27')

x = np.arange(len(image_names) + 1)
w = 0.35
names = [n[:18] for n in image_names] + ['OVERALL']

COLOR_TESS = '#e05c5c'
COLOR_VLM  = '#4fc3a1'

for ax, tess_vals, vlm_vals, metric in [
    (axes[0],
     per_sample_cer_tess + [tess_overall_cer],
     per_sample_cer_vlm  + [overall_cer],
     'CER'),
    (axes[1],
     per_sample_wer_tess + [tess_overall_wer],
     per_sample_wer_vlm  + [overall_wer],
     'WER'),
]:
    b1 = ax.bar(x - w/2, tess_vals, w, label='Tesseract',
                color=COLOR_TESS, alpha=0.9)
    b2 = ax.bar(x + w/2, vlm_vals,  w, label='VLM (Ours)',
                color=COLOR_VLM, alpha=0.9)

    # value labels
    for bar in list(b1) + list(b2):
        h = bar.get_height()
```

```
            ax.text(bar.get_x() + bar.get_width()/2, h + 0.03,
                    f'{h:.2f}', ha='center', fontsize=7,
                    color='white', fontweight='bold')

        # highlight overall
        for bar in [b1[-1], b2[-1]]:
            bar.set_edgecolor('white')
            bar.set_linewidth(1.5)

        ax.set_xticks(x)
        ax.set_xticklabels(names, rotation=35, ha='right',
                           fontsize=8, color='#cccccc')

        ax.set_ylabel(f'{metric} (lower is better)',
                      color='#cccccc', fontsize=10)

        ax.set_title(f'{metric} Comparison',
                     color='white', fontsize=12, fontweight='bold')

        ax.tick_params(colors='#cccccc')
        ax.grid(True, linestyle='--', alpha=0.3)
        ax.set_axisbelow(True)

        ax.legend(handles=[
            mpatches.Patch(color=COLOR_TESS,
                           label=f'Tesseract ({tess_overall_cer:.3f}/{tess_overall_wer:.3f})'),
            mpatches.Patch(color=COLOR_VLM,
                           label=f'VLM ({overall_cer:.3f}/{overall_wer:.3f})'),
        ], fontsize=8, facecolor='#1a1d27',
           labelcolor='white', edgecolor='#2a2d3a')

    # title
    plt.suptitle(
        f'17th-Century Spanish OCR Benchmark\n'
        f'CER ↓ {cer_improvement:.1f}%   |   WER ↓ {wer_improvement:.1f}%',
        color='white', fontsize=14, fontweight='bold'
    )

    # FIX layout warning
    plt.subplots_adjust(top=0.88, bottom=0.25)

    # save
    chart_path = OUT_DIR / 'final_comparison_chart.png'
    plt.savefig(chart_path, dpi=160, bbox_inches='tight',
                facecolor='#0f1117')

    # show
    plt.show()

    print(f"\n💾 Saved at: {chart_path}")

    # display saved
    display(Image(filename=str(chart_path)))
```

```
💾 Saved at: /content/drive/MyDrive/Saved_PDF_Data/Evaluation_Results/final_comparison_chart.png
```